Name – kashish chauhan I

Sec – B, Roll - 28

Experiment 7

Customer Segmentation using K-Means Clustering

Aim

To implement the K-Means Clustering algorithm on a real-world dataset for customer segmentation and analyze customer groups based on purchasing behavior and demographic characteristics.

GITHUB LINK: https://github.com/suyash84501/Mastery-in-Data-Analytics-and-Visualizations-and-Career-Advancement/blob/main/Experiment%207.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Upload CSV file
uploaded = files.upload()

# Read dataset
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving customer_segmentation_result.csv to customer_segmentation_result.csv
Dataset loaded successfully!
Shape: (20, 6)
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```
print("Dataset Information:")
df.info()

print("\nMissing Values:")
```

```
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())

print("\nStatistical Summary:")
display(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              20 non-null     int64
 1   Gender                  20 non-null     object
 2   Age                     20 non-null     int64
 3   Annual Income (k$)      20 non-null     int64
 4   Spending Score (1-100)  20 non-null     int64
 5   Cluster                 20 non-null     int64
dtypes: int64(5), object(1)
memory usage: 1.1+ KB

Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
Cluster                   0
dtype: int64

Duplicate Rows:
0

Statistical Summary:
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
count	20.00000	20.000000	20.000000	20.000000	20.000000
mean	10.50000	33.650000	18.800000	50.950000	2.300000
std	5.91608	15.114563	2.330575	34.149016	1.380313
min	1.00000	19.000000	15.000000	3.000000	0.000000
25%	5.75000	22.000000	17.000000	14.750000	1.000000
50%	10.50000	30.500000	19.000000	53.000000	3.000000
75%	15.25000	35.500000	20.000000	77.500000	3.000000
max	20.00000	67.000000	23.000000	99.000000	4.000000

```
# Remove duplicate rows
df = df.drop_duplicates()

# Remove rows containing missing values
df = df.dropna()

# Select important features
features = [
    'Age',
    'Annual Income (k$)',
    'Spending Score (1-100)'
]

X = df[features]

print("Selected Features:")
print(X.head())
```

```
print("\nShape after preprocessing:" , X.shape)
```

```
Selected Features:
   Age  Annual Income (k$)  Spending Score (1-100)
0   19                  15                      39
1   21                  15                      81
2   20                  16                       6
3   23                  16                      77
4   31                  17                      40

Shape after preprocessing: (20, 3)
```

```
# Standardize the features
scaler = StandardScaler ()

X_scaled = scaler.fit_transform (X)

# Convert into DataFrame for easy viewing
X_scaled_df = pd.DataFrame (
    X_scaled,
    columns=features
)

print("Scaled Data:" )
display(X_scaled_df.head ())
```

Scaled Data:

	Age	Annual Income (k$)	Spending Score (1-100)
0	-0.994444	-1.672857	-0.359028
1	-0.858684	-1.672857	0.902827
2	-0.926564	-1.232631	-1.350485
3	-0.722923	-1.232631	0.782650
4	-0.179882	-0.792406	-0.328984

```
wcss = []

# Calculate WCSS for different values of K
for k in range(1, 11):
    kmeans = KMeans (
        n_clusters=k ,
        random_state= 42,
        n_init= 10
    )

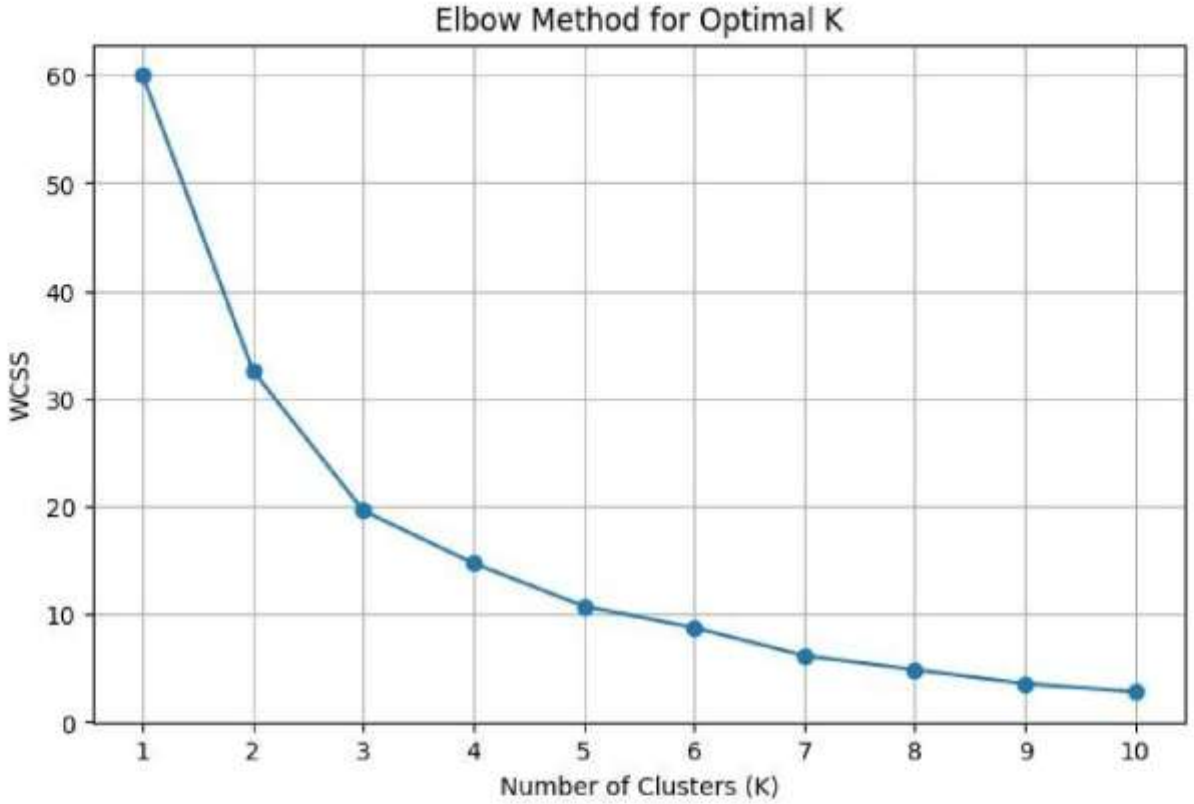
    kmeans.fit (X_scaled)
    wcss.append (kmeans.inertia_ )

# Plot Elbow Curve
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, 11),
    wcss,
    marker='o'
)

plt.title('Elbow Method for Optimal K' )
plt.xlabel('Number of Clusters (K)' )
plt.ylabel('WCSS')
plt.xticks(range(1, 11))
plt.grid(True)
```

```
plt.show()
```




```
silhouette_scores = []

# Calculate silhouette score for K = 2 to 10
for k in range(2, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = kmeans.fit_predict(X_scaled)

    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

    print(
        "K =", k,
        " Silhouette Score =",
        round(score, 4)
    )

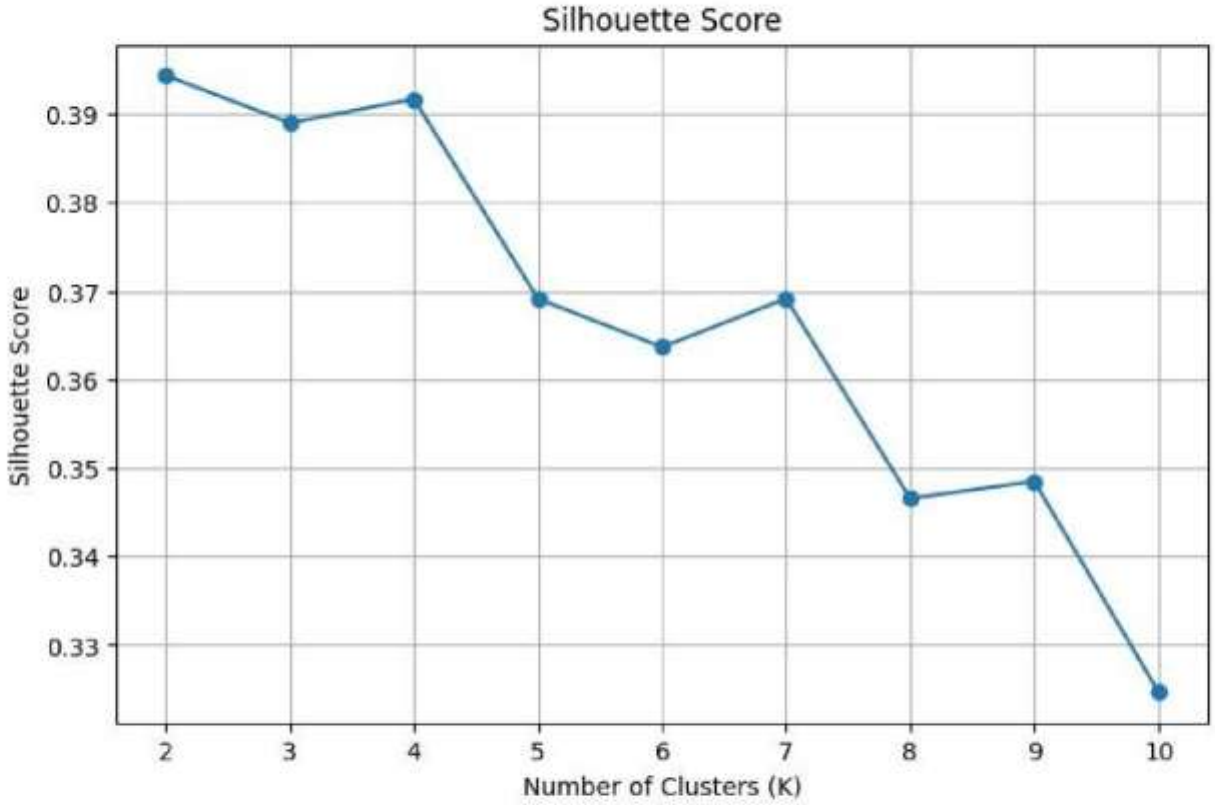
# Plot scores
plt.figure(figsize=(8, 5))

plt.plot(
    range(2, 11),
    silhouette_scores,
    marker='o'
)

plt.title('Silhouette Score')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score')
plt.xticks(range(2, 11))
plt.grid(True)

plt.show()
```

```
K = 2  Silhouette Score = 0.3944
K = 3  Silhouette Score = 0.389
K = 4  Silhouette Score = 0.3917
K = 5  Silhouette Score = 0.3691
K = 6  Silhouette Score = 0.3637
K = 7  Silhouette Score = 0.3691
K = 8  Silhouette Score = 0.3465
K = 9  Silhouette Score = 0.3485
K = 10  Silhouette Score = 0.3247
```




```
# Select number of clusters
k = 5

# Create and train K-Means model
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

# Predict cluster for each customer
df['Cluster'] = kmeans.fit_predict(X_scaled)

print("K-Means clustering completed!")
print("\nCustomer Cluster Distribution:")
print(df['Cluster'].value_counts().sort_index())

print("\nSample Data:")
display(df.head(10))
```

```
K-Means clustering completed!

Customer Cluster Distribution:
Cluster
0    3
1    3
2    3
3    7
4    4
Name: count, dtype: int64

Sample Data:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	3
8	9	Male	64	19	3	0
9	10	Female	30	19	72	3

```
# Cluster visualization
plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    s=100
)

# Plot cluster centroids
centers = scaler.inverse_transform(kmeans.cluster_centers_)

plt.scatter(
    centers[:, 1],
    centers[:, 2],
    marker='X',
    s=250,
    label='Centroids'
)

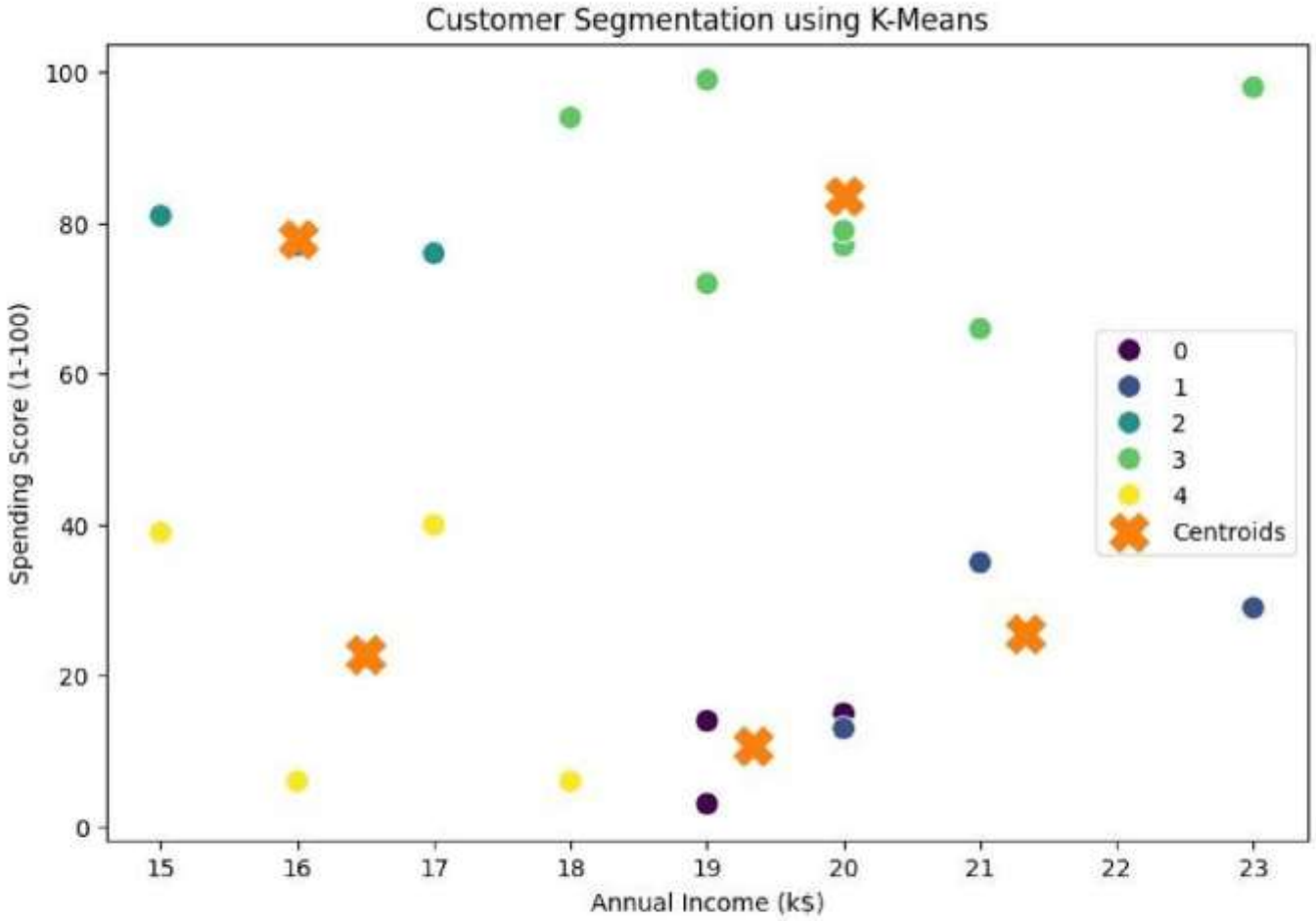
plt.title('Customer Segmentation using K-Means')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()

plt.show()

# Analyze clusters
print("\nAverage Characteristics of Each Cluster:")

cluster_summary = df.groupby('Cluster')[features].mean()
```

```
display(cluster_summary.round(2))
```



Average Characteristics of Each Cluster:

Cluster	Age	Annual Income (k$)	Spending Score (1-100)
0	63.00	19.33	10.67
1	41.33	21.33	25.67
2	22.00	16.00	78.00
3	27.00	20.00	83.57
4	26.25	16.50	22.75

Questions & Answers

Q1. What is clustering? How does it differ from classification?

Answer: Clustering is an unsupervised machine learning technique used to group similar data points into clusters. It does not require predefined labels. Classification is a supervised learning technique in which data is assigned to predefined classes using labelled training data.

Q2. Explain the working principle of the K-Means Clustering algorithm.

Answer: K-Means divides the dataset into a specified number of clusters (K). First, K centroids are selected randomly. Each data point is assigned to the nearest centroid, then the centroids are recalculated based on the mean of the assigned points. This process continues until the centroids become stable.

Q3. Why is feature scaling important before applying K-Means clustering?

Answer: Feature scaling makes all selected features contribute equally to clustering. K-Means uses distance calculations, so a feature with a larger numerical range can dominate the results. Standardization brings the features to a comparable scale and improves clustering performance.

Q4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

Answer: The Elbow Method is used to find a suitable value of K in K-Means. It calculates the Within-Cluster Sum of Squares (WCSS) for different numbers of clusters. The point where the decrease in WCSS starts becoming slower is considered the elbow and is generally selected as the optimal K.

Q5. What is the Silhouette Score? How is it used to evaluate clustering performance?

Answer: The Silhouette Score measures how well each data point fits within its assigned cluster compared with other clusters. Its value generally ranges from -1 to 1. A higher positive score indicates that the clusters are well separated and the data points are properly grouped.

Q6. Why is K-Means considered an unsupervised machine learning algorithm?

Answer: K-Means is considered unsupervised because it works without predefined class labels. It discovers natural groups or patterns in the dataset based on the similarity between data points. The clusters are created automatically according to the selected features.

Q7. Mention any four real-world applications of customer segmentation.

Answer: Four applications of customer segmentation are:

Targeted marketing campaigns

Personalized product recommendations

Customer retention and loyalty programs

Improving customer engagement

Customer segmentation helps businesses understand different customer groups and create suitable strategies for them

Q8. What are the limitations of the K-Means algorithm?

Answer: K-Means requires the number of clusters to be selected in advance. It can be affected by the initial selection of centroids and outliers. It also works best when clusters are relatively compact and well separated. Different feature scales can also affect the results if scaling is not performed.

Q9. How can businesses use customer segmentation to improve marketing and customer retention?

Answer: Businesses can identify groups of customers with similar needs, purchasing behavior, or spending patterns. They can then create targeted offers, personalized advertisements, loyalty programs, and product recommendations for each group. This can improve customer engagement and help retain customers.

Q10. Compare K-Means Clustering with Hierarchical Clustering based on their working principles and applications.

K-Means Clustering	Hierarchical Clustering
Divides data into a predefined number of clusters.	Builds a hierarchy of clusters.
Requires the value of K.	Does not require K at the beginning.
Uses centroids and distance calculations.	Uses distances to merge or split clusters.
Generally faster for large datasets.	Can be computationally expensive for large datasets.
Useful for customer segmentation and grouping large datasets.	Useful when hierarchical relationships between groups are important.